

Name: Pramod Pandit

Reg no: 12305901

1. Binary Tree Inorder Traversal

The screenshot shows the LeetCode interface for problem 94. The left panel contains the problem description: "Given the root of a binary tree, return the inorder traversal of its nodes' values." It includes an example with input root = [1,null,2,3] and output [1,3,2]. A diagram shows a node with value 1. The right panel shows the C++ code for an inorder traversal function. The code is as follows:

```
public:
    void inorder(TreeNode* root, vector<int> &ans) {
        if(root == NULL) return;
        inorder(root->left, ans);
        ans.push_back(root->val);
        inorder(root->right, ans);
    }

    vector<int> inorderTraversal(TreeNode* root) {
        vector<int> ans;
        inorder(root, ans);
        return ans;
    }
};
```

The status bar at the bottom indicates the solution is "Accepted" with a runtime of 0 ms.

2. Range Sum of BST

The screenshot shows the LeetCode interface for problem 938. The left panel contains the problem description: "Given the root node of a binary search tree and two integers low and high, return the sum of values of all nodes with a value in the inclusive range [low, high]." It includes an example with a tree diagram showing nodes 5, 10, and 15. The right panel shows the C++ code for a range sum function. The code is as follows:

```
int sum(TreeNode* root, int low, int high) {
    if(root == NULL) return 0;

    if(root->val < low) {
        return sum(root->right, low, high);
    }
    if(root->val > high) {
        return sum(root->left, low, high);
    }
    return root->val
        + sum(root->right, low, high)
        + sum(root->left, low, high);
}

int rangeSumBST(TreeNode* root, int low, int high) {
    return sum(root, low, high);
}
```

The status bar at the bottom indicates the solution is "Accepted" with a runtime of 0 ms.

3. Convert Sorted Array to Binary Search Tree

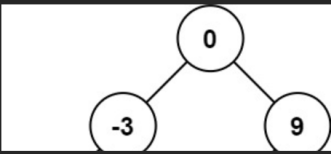
DescriptionAccepted ×EditorialSolutionsSubmissions

108. Convert Sorted Array to Binary Search Tree

EasyTopicsCompanies

Given an integer array `nums` where the elements are sorted in **ascending order**, convert it to a **height-balanced** binary search tree.

Example 1:



```
graph TD; 0((0)) --- -3((-3)); 0 --- 9((9))
```

12K20362 Online

Code

```
C++Auto
12 class Solution {
13 public:
14     TreeNode* buildBST(vector<int>& nums, int left, int right) {
15         if (left > right)
16             return NULL;
17
18         int mid = left + (right - left) / 2;
19
20         TreeNode* root = new TreeNode(nums[mid]);
21
22         root->left = buildBST(nums, left, mid - 1);
23
24         root->right = buildBST(nums, mid + 1, right);
25
26         return root;
27     }
28 }
```

SavedLn 14, Col 65

TestcaseTest Result

AcceptedRuntime: 0 ms

4. Increasing Order Search Tree

Problem List<>Submit

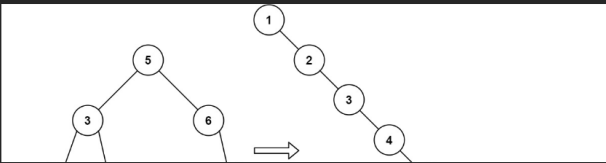
DescriptionAccepted ×EditorialSolutionsSubmissions

897. Increasing Order Search Tree

EasyTopicsCompanies

Given the `root` of a binary search tree, rearrange the tree in **in-order** so that the leftmost node in the tree is now the root of the tree, and every node has no left child and only one right child.

Example 1:



```
graph TD; 5((5)) --- 3((3)); 5 --- 6((6)); 3 --- 1((1)); 3 --- 2((2)); 6 --- 3 --- 4((4))
```

4.5K4511 Online

Code

```
C++Auto
12 class Solution {
13 public:
14     void inorder(TreeNode* root, vector<int>& ans) {
15         if (root == NULL)
16             return;
17
18         inorder(root->left, ans);
19         ans.push_back(root->val);
20         inorder(root->right, ans);
21     }
22
23     TreeNode* increasingBST(TreeNode* root) {
24         vector<int> ans;
25         inorder(root, ans);
26
27         TreeNode* newRoot = new TreeNode(ans[0]);
28     }
```

SavedLn 37, Col 3

TestcaseTest Result

AcceptedRuntime: 0 ms